

Brute Force - Sorting Algorithms

Selection Sort, Bubble Sort, Insertion Sort; compare execution times

Dr. Poongavanam N

Name: Y. Jithendra Reddy

Reg.no: 192525303

DAA Topic 02 Experiments

Sorting Algorithms — Worked Scenario

Questions

Selection Sort · Bubble Sort · Insertion Sort — 5 Solved Questions Each

1. Selection Sort

Question 1: Trophy Ranking System

A sports event organizer has recorded the scores of 50 participants but needs to display only the top 5 scores on a leaderboard screen. Since only a few top values are needed (not a full sort), use selection sort's 'find maximum repeatedly' approach to extract the top 5.

Approach

Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required. This avoids the cost of sorting all 50 values fully.

Complexity Analysis

Aspect Complexity Explanation

Time $O(k \times n)$ Only k passes are made instead of n , each scanning the remaining $n-i$ elements to find the max.

Space $O(1)$ Sorting is done in place; only a copy of the input array

is used for safety.

Test Cases

```
assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
```

```
assert top_k_scores([5,3,1], 5) == [5,3,1] # fewer than k items
```

```
assert top_k_scores([], 3) == [] # empty input
```

```
print('All test cases passed!')
```

A screenshot of a Python IDE interface. The left pane shows a file named 'main.py' with the following code:

```
1 def top_k(scores, k):
2     n = len(scores)
3
4     for i in range(min(k, n)):
5         max_index = i
6         for j in range(i + 1, n):
7             if scores[j] > scores[max_index]:
8                 max_index = j
9             scores[i], scores[max_index] = scores[max_index], scores[i]
10
11     return scores[:k]
12
13
14 n = int(input("Enter number of scores: "))
15 scores = list(map(int, input("Enter scores: ").split()))
16 k = int(input("Enter K: "))
17
18 print("Top Scores:", top_k(scores, k))
```

The right pane shows the input and output of the program:

```
Enter number of scores: 10
Enter scores: 72 88 65 90 77 95 60 83 91 68
Enter K: 5
Top Scores: [95, 91, 90, 88, 83]

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2: Memory-Constrained Embedded Device

A low-memory IoT sensor device collects temperature readings and must sort them with minimal write/swap operations, since each write to memory is costly on this hardware. Selection

sort performs at most $n-1$ swaps regardless of input order, making it preferable to bubble or insertion sort here. Implement it with a swap counter to prove the claim.

Approach

Use the standard selection sort algorithm, but only perform a swap when the found minimum is

not already in the correct position. Track the number of swaps performed and compare it against $n-1$ (the theoretical maximum) to demonstrate the low write cost.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$ Two nested loops compare every pair of elements

regardless of input order.

Writes $\leq n - 1$ A swap only occurs when the minimum found is not already in place, bounding total memory writes.

Test Cases

```
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
```

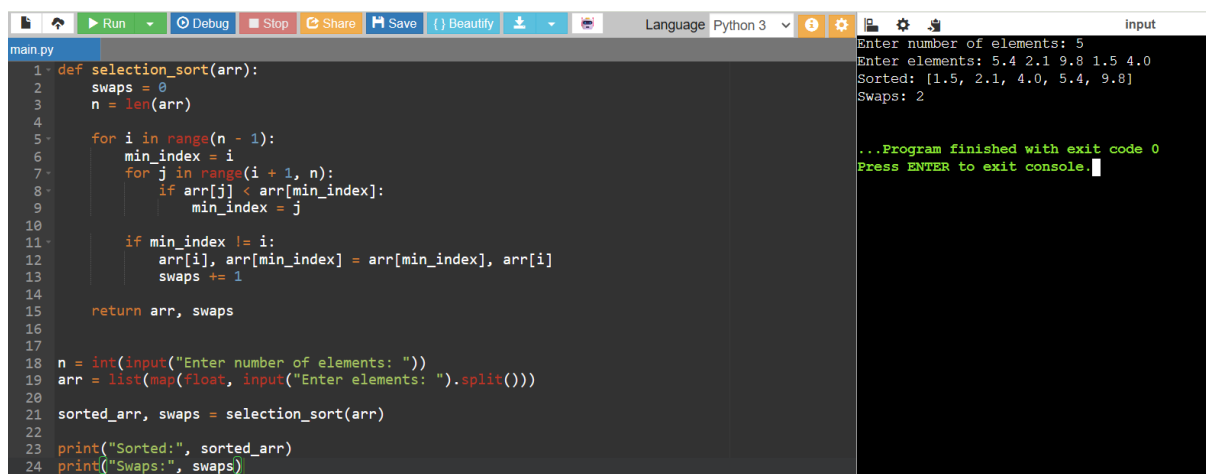
```
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
```

```
assert sw <= len(res) - 1 # write bound holds
```

```
res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
```

```
assert sw2 == 0 # already sorted -> zero swaps
```

```
print('All test cases passed!')
```



```
main.py
1 def selection_sort(arr):
2     swaps = 0
3     n = len(arr)
4
5     for i in range(n - 1):
6         min_index = i
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         if min_index != i:
12             arr[i], arr[min_index] = arr[min_index], arr[i]
13             swaps += 1
14
15     return arr, swaps
16
17
18 n = int(input("Enter number of elements: "))
19 arr = list(map(float, input("Enter elements: ").split()))
20
21 sorted_arr, swaps = selection_sort(arr)
22
23 print("Sorted:", sorted_arr)
24 print("Swaps:", swaps)
```

input

```
Enter number of elements: 5
Enter elements: 5.4 2.1 9.8 1.5 4.0
Sorted: [1.5, 2.1, 4.0, 5.4, 9.8]
Swaps: 2

...Program finished with exit code 0
Press ENTER to exit console
```

Question 3: Library Book Reordering

A librarian wants to rearrange a shelf of 20 books by ID number, but physically moving a book (swap) is time-consuming, while checking labels (comparison) is fast. Model this as an array sorting problem using selection sort and report the number of physical "moves" needed.

Approach

Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are

unrestricted since they are cheap.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$ Comparisons dominate the runtime; for $n = 20$ this is

190 comparisons.

Physical Moves $\leq n - 1$ Each shelf position is corrected with at most one

physical swap.

Test Cases

```
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
```

```
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
```

```
assert moves <= len(ordered) - 1
```

```
ordered2, moves2 = reorder_shelf([100, 200, 300]) # already sorted
```

```
assert moves2 == 0
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a selection sort algorithm to reorder an array of book IDs. The function 'reorder_books(arr)' takes an array and returns the sorted array and the number of swaps (moves). The main program prompts the user for the number of books and their IDs, then calls the function and prints the results.

```
1 def reorder_books(arr):
2     moves = 0
3     n = len(arr)
4
5     for i in range(n - 1):
6         min_index = i
7
8         for j in range(i + 1, n):
9             if arr[j] < arr[min_index]:
10                min_index = j
11
12            if min_index != i:
13                arr[i], arr[min_index] = arr[min_index], arr[i]
14                moves += 1
15
16        return arr, moves
17
18
19 n = int(input("Enter number of books: "))
20 books = list(map(int, input("Enter book IDs: ").split()))
21
22 sorted_books, moves = reorder_books(books)
23
24 print("Sorted Book IDs:", sorted_books)
25 print("Physical Moves:", moves)
```

The output on the right shows the user inputting 6 books with IDs 105, 102, 108, 101, 110, 103. The program outputs the sorted IDs [101, 102, 103, 105, 108, 110] and 3 physical moves. It also shows a message: "...Program finished with exit code 0 Press ENTER to exit console."

Question 4: Contest Prize Distribution

In a coding contest, you must repeatedly select the participant with the current highest score, print their name, remove them from the list, and repeat until all participants are ranked.

Implement this using selection sort logic without using a separate sorting library.

Approach

Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as

the next-ranked winner.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$ Each of the n ranking positions requires a full scan of

the remaining participants.

Space $O(1)$ extra Ranking is produced in place using swaps; the output

list just records the order.

Test Cases

```
ranking = distribute_prizes([(Asha,88), (Ravi,95),  
(Meera,79), (Dev,95)])
```

```
scores_only = [p[1] for p in ranking]
```

```
assert scores_only == sorted(scores_only, reverse=True) # descending order
```

```
assert ranking[0][1] == 95 # top score first
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a selection sort algorithm to sort a list of participants by their scores. The program prompts the user to enter the number of participants (3), then for each participant, it prompts for a name and a score. The scores are: Ravi (95), Sita (92), and Arun (85). The program then prints the ranking, which is Sita - 92, Arun - 85, and Ravi - 78. The output also shows that the program finished with exit code 0.

```
main.py
1 n = int(input("Enter number of participants: "))
2
3 players = []
4
5 for i in range(n):
6     name = input("Enter name: ")
7     score = int(input("Enter score: "))
8     players.append([name, score])
9
10 for i in range(n):
11     max_index = i
12     for j in range(i + 1, n):
13         if players[j][1] > players[max_index][1]:
14             max_index = j
15
16     players[i], players[max_index] = players[max_index], players[i]
17
18 print("\nRanking")
19 for p in players:
20     print(p[0], "-", p[1])
```

input

```
Enter number of participants: 3
Enter name: RAVI
Enter score: 78
Enter name: SITA
Enter score: 92
Enter name: ARUN
Enter score: 85

Ranking
SITA - 92
ARUN - 85
RAVI - 78

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5: Small Dataset in a Mobile App

A shopping app needs to sort a “recently viewed” list of just 8–10 items by price whenever the

user opens the screen. Argue whether selection sort is a reasonable choice for such a small, frequently-refreshed list, and implement it with timing to support your answer.

Approach

For very small n , the $O(n^2)$ cost of selection sort is negligible in absolute terms (a handful of microseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain

choice rather than pulling in a general-purpose sort for 8–10 items.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$, $n \leq 10$ For such small n , absolute execution time is in the microsecond range, so complexity class is not a

practical concern.

Space $O(1)$ In-place sort, no extra memory beyond the working

copy.

Test Cases

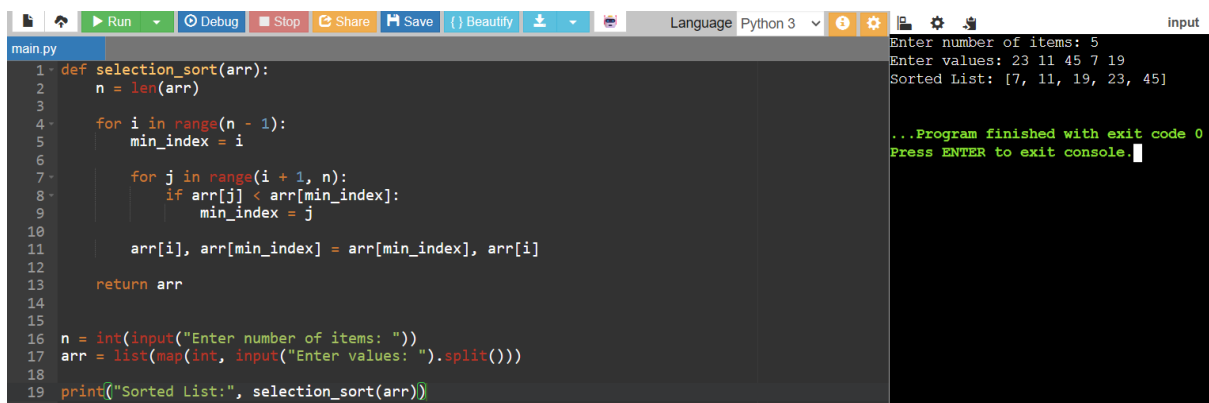
```
assert selection_sort([499,129,899,45,275,60,310,150]) ==
```

```
sorted([499,129,899,45,275,60,310,150])
```

```
assert selection_sort([]) == [] # empty list
```

```
assert selection_sort([7]) == [7] # single item
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a `selection_sort` function and a main block that takes user input. The function iterates through the array, finding the minimum element in each pass and swapping it with the element at the current index. The main block prompts the user for the number of items and their values, then prints the sorted list. The output on the right shows the user inputting 5 items with values 23, 11, 45, 7, and 19, resulting in a sorted list of [7, 11, 19, 23, 45].

```
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n - 1):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11         arr[i], arr[min_index] = arr[min_index], arr[i]
12
13     return arr
14
15
16 n = int(input("Enter number of items: "))
17 arr = list(map(int, input("Enter values: ").split()))
18
19 print("Sorted List:", selection_sort(arr))
```

Enter number of items: 5
Enter values: 23 11 45 7 19
Sorted List: [7, 11, 19, 23, 45]
...Program finished with exit code 0
Press ENTER to exit console.

2. Bubble Sort

Question 1: Exam Result Slip Correction

A teacher has a nearly sorted list of student roll numbers (only 2–3 are out of place due to late corrections). Use optimized bubble sort (with an early-exit flag) to fix the order and show it takes

very few passes.

Approach

Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass

completes with no swaps, the list is already sorted and the algorithm exits early — ideal for

nearly sorted data like this.

Complexity Analysis

Aspect Complexity Explanation

Best Case $O(n)$ One pass detects a fully sorted array and exits when

no swaps occur.

This Scenario $\approx O(n)$ Only 2–3 misplaced roll numbers means very few passes are needed before the flag stays false.

Test Cases

`sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])`


```

assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])

assert passes < 8 # fewer than full n passes

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5]) # already sorted

assert passes_ok == 1 # exits after first pass

print('All test cases passed!')

```

```

main.py
1 def optimized_bubble_sort(arr):
2     n = len(arr)
3     passes = 0
4
5     for i in range(n - 1):
6         swapped = False
7         passes += 1
8
9         for j in range(n - 1 - i):
10            if arr[j] > arr[j + 1]:
11                arr[j], arr[j + 1] = arr[j + 1], arr[j]
12                swapped = True
13
14            if not swapped:
15                break
16
17        return arr, passes
18
19
20 n = int(input("Enter number of roll numbers: "))
21 roll_numbers = list(map(int, input("Enter roll numbers: ").split()))
22
23 if len(roll_numbers) != n:
24     print("Invalid number of inputs")
25 else:
26     sorted_rolls, passes = optimized_bubble_sort(roll_numbers)
27     print("Sorted Roll Numbers:", sorted_rolls)
28     print("Number of Passes:", passes)

```

```

Input
Enter number of roll numbers: 8
Enter roll numbers: 101 102 104 103 105 107 106 108
Sorted Roll Numbers: [101, 102, 103, 104, 105, 106, 107, 108]
Number of Passes: 2

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 2: Traffic Signal Priority Queue

At an intersection, vehicle priorities (ambulance > bus > car) arrive in a small queue and need

constant reordering as new vehicles arrive. Simulate this using bubble sort each time a new vehicle is added, and discuss whether bubble sort's simplicity outweighs its inefficiency here.

Approach

Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's $O(n^2)$ cost is negligible, and its

simplicity makes it easy to reason about and verify in a safety-critical system.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$ Re-sorting the whole small queue on every arrival is acceptable since n stays very small.

Trade-off Simplicity vs.
efficiency

Bubble sort is easy to verify for correctness in a safety-critical setting, which outweighs its inefficiency

at this scale.

Test Cases

```
queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
assert result == ['ambulance', 'bus', 'car', 'car']

assert bubble_sort_queue(['ambulance']) == ['ambulance'] # single
vehicle
print('All test cases passed!')
```



```
main.py
1 def bubble_sort_queue(queue):
2     priority = {
3         "ambulance": 3,
4         "bus": 2,
5         "car": 1
6     }
7
8     n = len(queue)
9
10    for i in range(n - 1):
11        swapped = False
12
13        for j in range(n - 1 - i):
14            if priority[queue[j].lower()] < priority[queue[j + 1].lower()]:
15                queue[j], queue[j + 1] = queue[j + 1], queue[j]
16                swapped = True
17
18        if not swapped:
19            break
20
21    return queue
22
23
24    n = int(input("Enter number of vehicles: "))
25    vehicles = []
26
27    for i in range(n):
28        vehicle = input("Enter vehicle type: ").lower()
29
30        if vehicle not in ["ambulance", "bus", "car"]:
31            print("Invalid vehicle type")
32            break
33
34        vehicles.append(vehicle)
35
36    if len(vehicles) == n:
37        print("Priority Queue:", bubble_sort_queue(vehicles))

```

Enter number of vehicles: 4
Enter vehicle type: CAR
Enter vehicle type: CAR
Enter vehicle type: BUS
Enter vehicle type: AMBULANCE
Priority Queue: ['ambulance', 'bus', 'car', 'car']

...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Bubble Sort Visualization Tool

You are building an educational tool to visually demonstrate how “bubbles” (largest elements)

rise to the top with each pass. Implement bubble sort so it prints/returns the state of the array

after every single pass, useful for animation.

Approach

Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through

UI.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n^2)$ Same as standard bubble sort; storing frames adds

$O(n)$ work per pass.

Space $O(n^2)$ worst case Up to n frames are stored, each of size n , useful for

driving an animation.

Test Cases

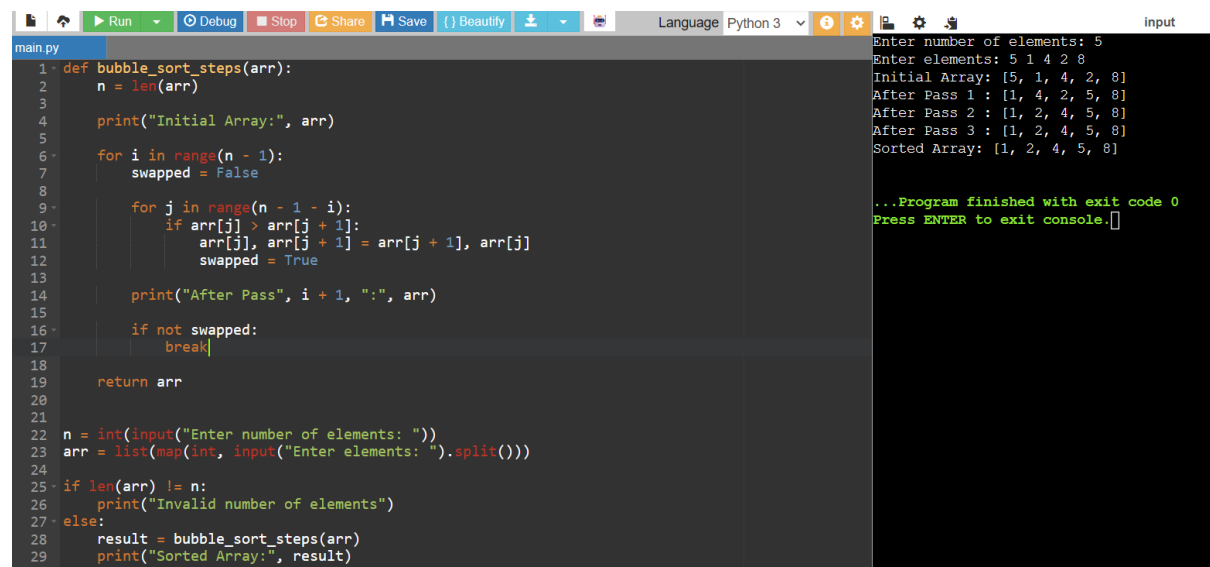
```
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
```

```
assert frames[-1] == sorted([5, 1, 4, 2, 8]) # final frame is sorted
```

```
assert frames[0] == [5, 1, 4, 2, 8] # first frame is the original input
```

```
assert len(frames) >= 2 # at least an initial and final frame
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a bubble sort algorithm with a function 'bubble_sort_steps' and a main execution block. The output console shows the program's execution for an input array [5, 1, 4, 2, 8], displaying the initial array, the state after each of three passes, and the final sorted array [1, 2, 4, 5, 8]. The program ends with 'Exit code 0'.

```
1 def bubble_sort_steps(arr):
2     n = len(arr)
3
4     print("Initial Array:", arr)
5
6     for i in range(n - 1):
7         swapped = False
8
9         for j in range(n - 1 - i):
10            if arr[j] > arr[j + 1]:
11                arr[j], arr[j + 1] = arr[j + 1], arr[j]
12                swapped = True
13
14            print("After Pass", i + 1, ":", arr)
15
16        if not swapped:
17            break
18
19    return arr
20
21
22 n = int(input("Enter number of elements: "))
23 arr = list(map(int, input("Enter elements: ").split()))
24
25 if len(arr) != n:
26     print("Invalid number of elements")
27 else:
28     result = bubble_sort_steps(arr)
29     print("Sorted Array:", result)
```

Enter number of elements: 5
Enter elements: 5 1 4 2 8
Initial Array: [5, 1, 4, 2, 8]
After Pass 1 : [1, 4, 2, 5, 8]
After Pass 2 : [1, 2, 4, 5, 8]
After Pass 3 : [1, 2, 4, 5, 8]
Sorted Array: [1, 2, 4, 5, 8]
...Program finished with exit code 0
Press ENTER to exit console.

Question 4: Sorting Sensor Alerts by Severity

A monitoring dashboard receives 15 alerts per minute that must be sorted by severity level (1 =

critical to 5 = low) for display. Use bubble sort and measure whether the early-termination

optimization meaningfully improves performance on this small, frequently-updated dataset.

Approach

Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the

optimization helps for this small, often-partially-sorted stream.

Complexity Analysis

Aspect Complexity Explanation

Plain Bubble Sort $O(n^2)$ always Always runs the full $n-1$ passes regardless of how sorted the data already is.

Optimized Bubble

Sort

$O(n)$ best, $O(n^2)$

worst

Exits early once a pass makes no swaps, which helps when alerts arrive already close to sorted.

Test Cases

```
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
```

```
r1, c1 = bubble_sort_plain(alerts)
```

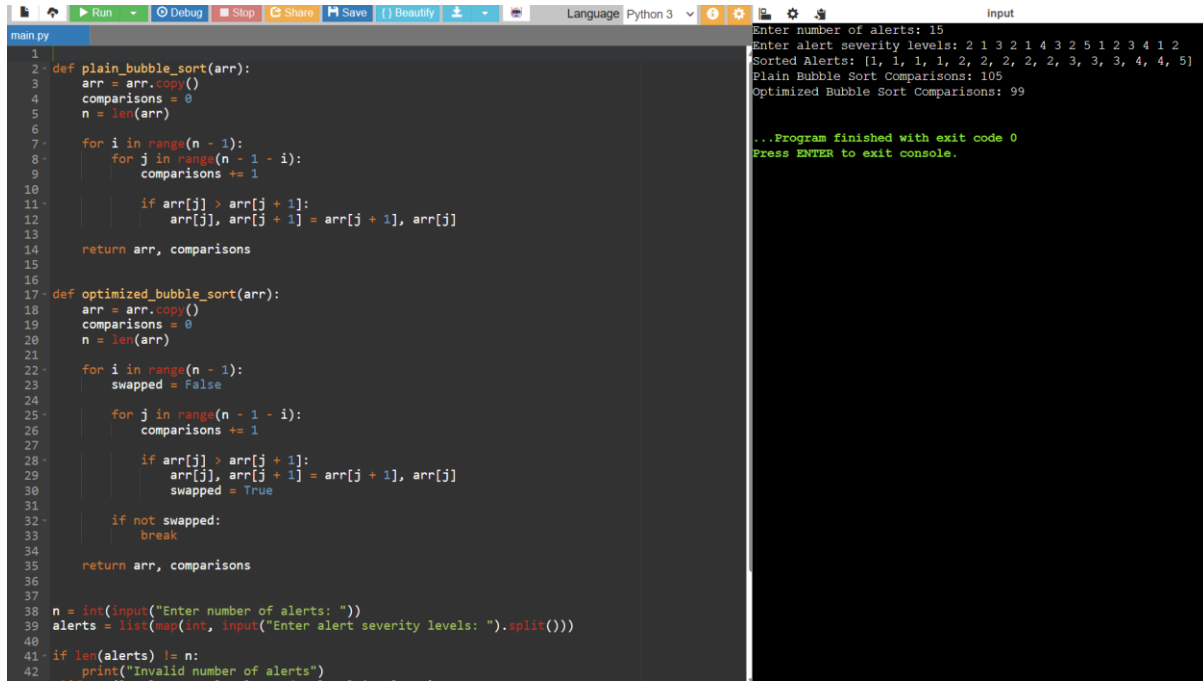
```
r2, c2 = bubble_sort_optimized(alerts)
```

```
assert r1 == r2 == sorted(alerts) # both produce the same sorted result
```

```
assert c2 <= c1 # optimized never does more comparisons
```

```
print('Plain comparisons:', c1, 'Optimized comparisons:', c2)
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code implements two bubble sort functions: 'plain_bubble_sort' and 'optimized_bubble_sort'. The 'plain_bubble_sort' function uses a nested loop to compare and swap elements, while the 'optimized_bubble_sort' function includes a 'swapped' flag to break the loop early if no swaps occur. The main code prompts the user for the number of alerts and their severity levels, then compares the results of both sorting functions. The output on the right shows the sorted alerts and the comparison counts for both methods.

```
1 def plain_bubble_sort(arr):
2     arr = arr.copy()
3     comparisons = 0
4     n = len(arr)
5
6     for i in range(n - 1):
7         for j in range(n - 1 - i):
8             comparisons += 1
9
10            if arr[j] > arr[j + 1]:
11                arr[j], arr[j + 1] = arr[j + 1], arr[j]
12
13        return arr, comparisons
14
15
16 def optimized_bubble_sort(arr):
17     arr = arr.copy()
18     comparisons = 0
19     n = len(arr)
20
21     for i in range(n - 1):
22         swapped = False
23         for j in range(n - 1 - i):
24             comparisons += 1
25
26             if arr[j] > arr[j + 1]:
27                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
28                 swapped = True
29
30         if not swapped:
31             break
32
33     return arr, comparisons
34
35
36 n = int(input("Enter number of alerts: "))
37 alerts = list(map(int, input("Enter alert severity levels: ").split()))
38
39 if len(alerts) != n:
40     print("Invalid number of alerts")
41
42
```

Enter number of alerts: 15
Enter alert severity levels: 2 1 3 2 1 4 3 2 5 1 2 3 4 1 2
Sorted Alerts: [1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 5]
Plain Bubble Sort Comparisons: 105
Optimized Bubble Sort Comparisons: 99
...Program finished with exit code 0
Press ENTER to exit console.

Question 5: Card Game Hand Sorting

In a card game app, a player's hand of 13 cards must be sorted by rank whenever a new card is

drawn. Since only one card is added at a time (nearly sorted data), implement bubble sort and

compare its pass count to a full re-sort from scratch.

Approach

Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel)

compared to sorting the entire hand from an unsorted state.

Complexity Analysis

Aspect Complexity Explanation

Nearly Sorted

Hand

$\approx O(n)$ Only the newly added card needs to “bubble” into place, so few passes are needed.

Fully Shuffled

Hand

$O(n^2)$ A randomly shuffled hand requires close to the maximum number of passes and swaps.

Test Cases

```
hand = [2, 4, 6, 8, 9, 11, 13]
```

```
hand.append(7) # nearly sorted: one new card
```

```
final_hand, passes_incremental = bubble_sort_hand(hand)
```

```
assert final_hand == sorted(hand)
```

```
import random
```

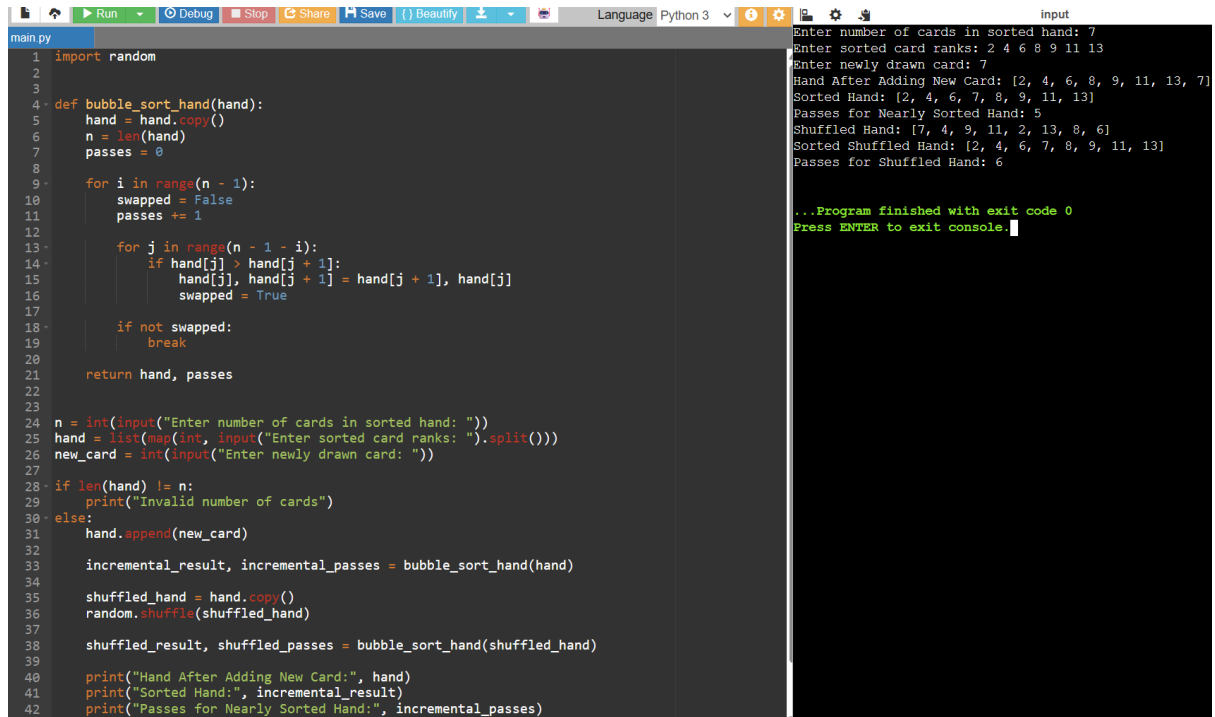
```
shuffled = hand.copy()
```

```
random.shuffle(shuffled)
```

```
_, passes_full = bubble_sort_hand(shuffled)
```

```
assert passes_incremental <= passes_full # nearly-sorted needs no more passes
```

```
print('All test cases passed!')
```



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a bubble sort function and a main program that interacts with the user. The output console shows the program's execution with user input and the resulting sorted and shuffled hands.

```
1 import random
2
3
4 def bubble_sort_hand(hand):
5     hand = hand.copy()
6     n = len(hand)
7     passes = 0
8
9     for i in range(n - 1):
10         swapped = False
11         passes += 1
12
13         for j in range(n - 1 - i):
14             if hand[j] > hand[j + 1]:
15                 hand[j], hand[j + 1] = hand[j + 1], hand[j]
16                 swapped = True
17
18         if not swapped:
19             break
20
21     return hand, passes
22
23
24 n = int(input("Enter number of cards in sorted hand: "))
25 hand = list(map(int, input("Enter sorted card ranks: ").split()))
26 new_card = int(input("Enter newly drawn card: "))
27
28 if len(hand) != n:
29     print("Invalid number of cards")
30 else:
31     hand.append(new_card)
32
33     incremental_result, incremental_passes = bubble_sort_hand(hand)
34
35     shuffled_hand = hand.copy()
36     random.shuffle(shuffled_hand)
37
38     shuffled_result, shuffled_passes = bubble_sort_hand(shuffled_hand)
39
40     print("Hand After Adding New Card:", hand)
41     print("Sorted Hand:", incremental_result)
42     print("Passes for Nearly Sorted Hand:", incremental_passes)
```

input

```
Enter number of cards in sorted hand: 7
Enter sorted card ranks: 2 4 6 8 9 11 13
Enter newly drawn card: 7
Hand After Adding New Card: [2, 4, 6, 8, 9, 11, 13, 7]
Sorted Hand: [2, 4, 6, 7, 8, 9, 11, 13]
Passes for Nearly Sorted Hand: 5
Shuffled Hand: [7, 4, 9, 11, 2, 13, 8, 6]
Sorted Shuffled Hand: [2, 4, 6, 7, 8, 9, 11, 13]
Passes for Shuffled Hand: 6

...Program finished with exit code 0
Press ENTER to exit console.
```


3. Insertion Sort

Question 1: Live Leaderboard Updates

An online game updates a leaderboard every time a single player's score changes, while the rest of the list remains sorted. Use insertion sort to insert just the updated score into its correct position instead of re-sorting the entire list, and measure the efficiency gain.

Approach

Remove the updated player's old entry, then use the insertion-sort technique of shifting elements to find and insert the new score into its correct position in the already-sorted

leaderboard, avoiding a full re-sort.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n)$ Only the shift needed to place the single new score is performed, instead of an $O(n \log n)$ or $O(n^2)$ full re-

sort.

Space $O(1)$ extra Insertion happens via in-place shifting.

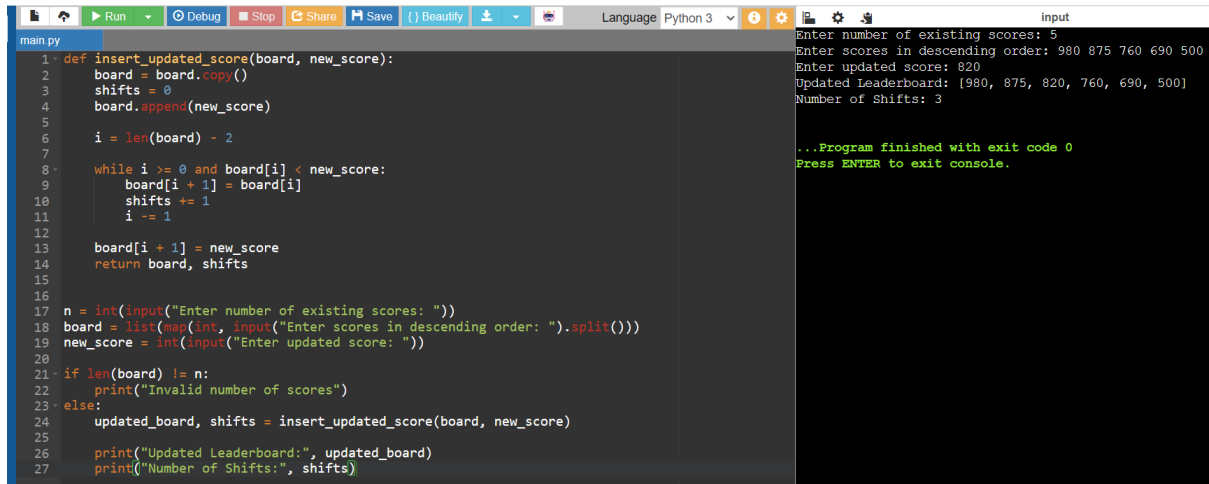
Test Cases

```
board = [980, 875, 760, 690, 500]
```

```
updated_board, shifts = insert_updated_score(board, 820)
```

```
assert updated_board == [980, 875, 820, 760, 690, 500]
```

```
board2, shifts2 = insert_updated_score(board, 100) # lowest score
assert board2[-1] == 100 and shifts2 == 0 # no shifts needed, goes to the end
print('All test cases passed!')
```



```
main.py
1 def insert_updated_score(board, new_score):
2     board = board.copy()
3     shifts = 0
4     board.append(new_score)
5
6     i = len(board) - 2
7
8     while i >= 0 and board[i] < new_score:
9         board[i + 1] = board[i]
10        shifts += 1
11        i -= 1
12
13    board[i + 1] = new_score
14    return board, shifts
15
16
17 n = int(input("Enter number of existing scores: "))
18 board = list(map(int, input("Enter scores in descending order: ").split()))
19 new_score = int(input("Enter updated score: "))
20
21 if len(board) != n:
22     print("Invalid number of scores")
23 else:
24     updated_board, shifts = insert_updated_score(board, new_score)
25
26     print("Updated Leaderboard:", updated_board)
27     print("Number of Shifts:", shifts)
```

```
input
Enter number of existing scores: 5
Enter scores in descending order: 980 875 760 690 500
Enter updated score: 820
Updated Leaderboard: [980, 875, 820, 760, 690, 500]
Number of Shifts: 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2: Card Sorting While Playing

A card player sorts their hand as they pick up cards one at a time (the classic real-world analogy for insertion sort). Implement this behavior programmatically, sorting a growing hand card by card.

Approach

Maintain a hand list. Each time a new card is picked up, insert it into its correct position among the already-sorted cards by shifting larger cards one position to the right, exactly like insertion

sort's inner loop.

Complexity Analysis

Aspect Complexity Explanation

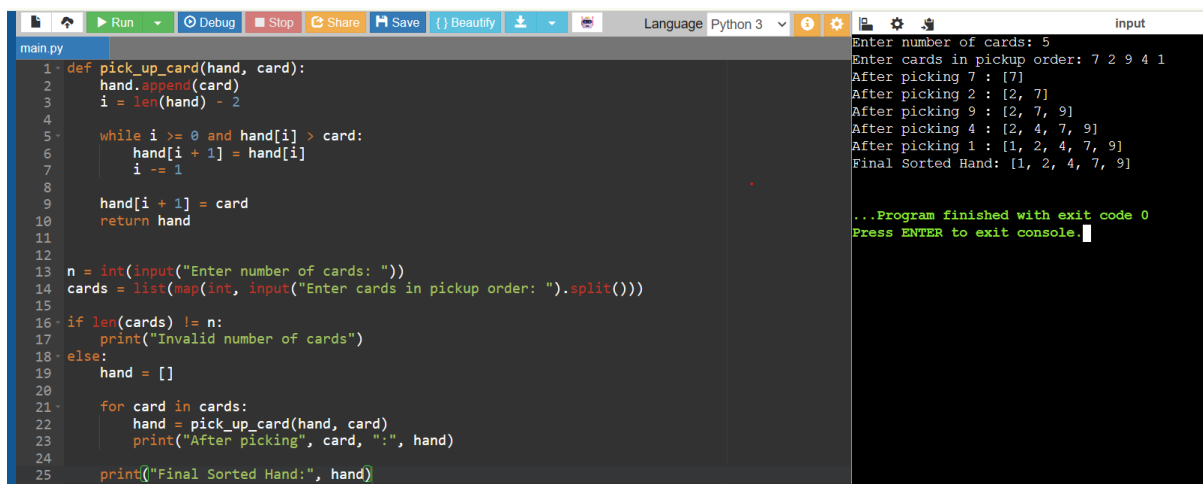
Time $O(n)$ per card,

$O(n^2)$ total

Each new card may require shifting past several existing cards; over n cards this totals $O(n^2)$.
Space $O(1)$ extra Cards are shifted in place within the hand list.

Test Cases

```
hand = []  
  
for card in [7, 2, 9, 4, 1]:  
    hand = pick_up_card(hand, card)  
  
assert hand == sorted([7, 2, 9, 4, 1])  
  
assert pick_up_card([], 5) == [5] # first card picked up  
  
print('All test cases passed!')
```



```
main.py  
1 def pick_up_card(hand, card):  
2     hand.append(card)  
3     i = len(hand) - 2  
4  
5     while i >= 0 and hand[i] > card:  
6         hand[i + 1] = hand[i]  
7         i -= 1  
8  
9     hand[i + 1] = card  
10    return hand  
11  
12  
13    n = int(input("Enter number of cards: "))  
14    cards = list(map(int, input("Enter cards in pickup order: ").split()))  
15  
16    if len(cards) != n:  
17        print("Invalid number of cards")  
18    else:  
19        hand = []  
20  
21        for card in cards:  
22            hand = pick_up_card(hand, card)  
23            print("After picking", card, ":", hand)  
24  
25    print("Final Sorted Hand:", hand)  
  
input  
Enter number of cards: 5  
Enter cards in pickup order: 7 2 9 4 1  
After picking 7 : [7]  
After picking 2 : [2, 7]  
After picking 9 : [2, 7, 9]  
After picking 4 : [2, 4, 7, 9]  
After picking 1 : [1, 2, 4, 7, 9]  
Final Sorted Hand: [1, 2, 4, 7, 9]  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Question 3: Real-Time Stock Price Feed

A trading dashboard receives stock prices one at a time and must maintain a sorted list at all times for quick min/max lookups. Use insertion sort to insert each new incoming price efficiently

into the already-sorted list.

Approach

Keep a running sorted list of prices. For each incoming price, use binary search (or a linear scan) to find its correct position, then shift subsequent elements right by one and insert it — the

core idea of insertion sort applied incrementally.

Complexity Analysis

Aspect	Complexity	Explanation
Time (binary search variant)	$O(\log n)$ search + $O(n)$ shift	Finding the position is fast, but shifting array elements to make room is still $O(n)$ per insertion.
Overall for n prices	$O(n^2)$ worst case	Still quadratic overall due to shifting, but far better than re-sorting the whole list from scratch each time.

Test Cases

prices = []

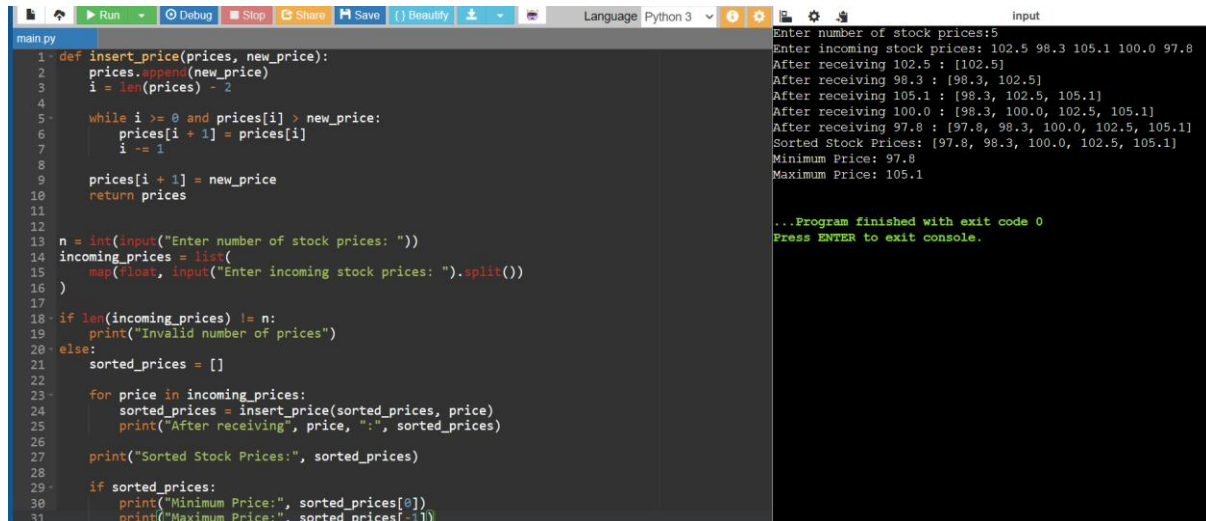
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:

prices = insert_price(prices, p)

assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])

assert prices[0] == min(prices) and prices[-1] == max(prices) # O(1) min/max lookup

print('All test cases passed!')



```
main.py
1 def insert_price(prices, new_price):
2     prices.append(new_price)
3     i = len(prices) - 2
4
5     while i >= 0 and prices[i] > new_price:
6         prices[i + 1] = prices[i]
7         i -= 1
8
9     prices[i + 1] = new_price
10    return prices
11
12
13 n = int(input("Enter number of stock prices: "))
14 incoming_prices = list(
15     map(float, input("Enter incoming stock prices: ").split())
16 )
17
18 if len(incoming_prices) != n:
19     print("Invalid number of prices")
20 else:
21     sorted_prices = []
22
23     for price in incoming_prices:
24         sorted_prices = insert_price(sorted_prices, price)
25         print("After receiving", price, ":", sorted_prices)
26
27     print("Sorted Stock Prices:", sorted_prices)
28
29     if sorted_prices:
30         print("Minimum Price:", sorted_prices[0])
31         print("Maximum Price:", sorted_prices[-1])
```

Input

Enter number of stock prices:5
Enter incoming stock prices: 102.5 98.3 105.1 100.0 97.8
After receiving 102.5 : [102.5]
After receiving 98.3 : [98.3, 102.5]
After receiving 105.1 : [98.3, 102.5, 105.1]
After receiving 100.0 : [98.3, 100.0, 102.5, 105.1]
After receiving 97.8 : [97.8, 98.3, 100.0, 102.5, 105.1]
Sorted Stock Prices: [97.8, 98.3, 100.0, 102.5, 105.1]
Minimum Price: 97.8
Maximum Price: 105.1
...Program finished with exit code 0
Press ENTER to exit console.

Question 4: Playlist Reordering by Recently Added

A music app lets users insert a newly added song into a playlist that is already sorted by duration. Implement insertion sort logic to place just the new song correctly instead of sorting

the whole playlist again.

Approach

Append the new song's duration to the playlist and shift it leftward past any longer songs until it

reaches its correct sorted position, exactly as the inner loop of insertion sort does for a single

new element.

Complexity Analysis

Aspect Complexity Explanation

Time $O(n)$ Only the new song is shifted into place among

already-sorted songs.

Space $O(1)$ extra Insertion is performed via in-place shifting within the

list.

Test Cases

```
playlist = [(&#39;Intro&#39;, 120), (&#39;Chill Beat&#39;, 210), (&#39;Long Jam&#39;, 340)]
```

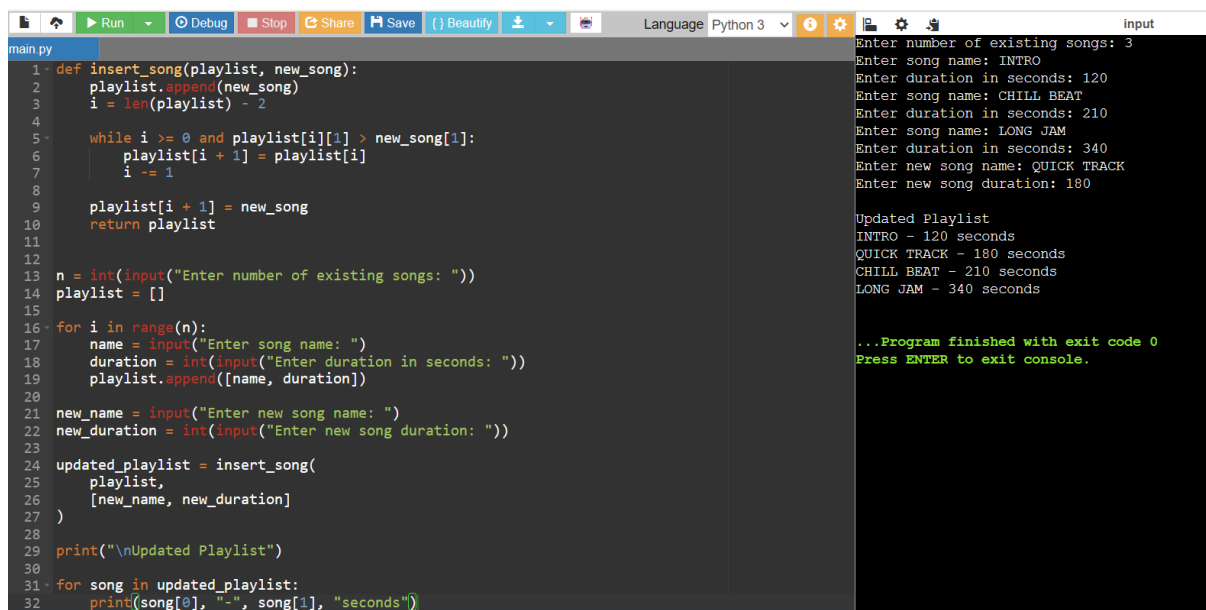
```
updated_playlist = insert_song(playlist, (&#39;Quick Track&#39;, 180))
```

```
durations = [s[1] for s in updated_playlist]
```

```
assert durations == sorted([120, 210, 340, 180])
```

```
assert (&#39;Quick Track&#39;, 180) in updated_playlist
```

```
print(&#39;All test cases passed!&#39;)
```



```
main.py
1 def insert_song(playlist, new_song):
2     playlist.append(new_song)
3     i = len(playlist) - 2
4
5     while i >= 0 and playlist[i][1] > new_song[1]:
6         playlist[i + 1] = playlist[i]
7         i -= 1
8
9     playlist[i + 1] = new_song
10    return playlist
11
12 n = int(input("Enter number of existing songs: "))
13 playlist = []
14
15 for i in range(n):
16     name = input("Enter song name: ")
17     duration = int(input("Enter duration in seconds: "))
18     playlist.append([name, duration])
19
20 new_name = input("Enter new song name: ")
21 new_duration = int(input("Enter new song duration: "))
22
23 updated_playlist = insert_song(
24     playlist,
25     [new_name, new_duration]
26 )
27
28 print("\nUpdated Playlist")
29
30 for song in updated_playlist:
31     print(song[0], "-", song[1], "seconds")
32
```

input

```
Enter number of existing songs: 3
Enter song name: INTRO
Enter duration in seconds: 120
Enter song name: CHILL BEAT
Enter duration in seconds: 210
Enter song name: LONG JAM
Enter duration in seconds: 340
Enter new song name: QUICK TRACK
Enter new song duration: 180

Updated Playlist
INTRO - 120 seconds
QUICK TRACK - 180 seconds
CHILL BEAT - 210 seconds
LONG JAM - 340 seconds

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5: Nearly Sorted Sensor Log Correction

A weather station logs temperature readings in mostly chronological order, but occasional delayed transmissions cause a few entries to be out of place. Use insertion sort to fix the log

and show that the number of shifts is very low compared to a randomly shuffled version of the

same data.

Approach

Run standard insertion sort on the log and count total shifts performed. Because only a few entries are out of place, most elements require zero or one shift, resulting in a shift count close

to the best-case $O(n)$ rather than the worst-case $O(n^2)$.

Complexity Analysis

Aspect Complexity Explanation

Nearly Sorted

Data

$\approx O(n)$ Very few shifts are needed since most elements are already close to their correct position.

Randomly

Shuffled Data

$O(n^2)$ Requires close to the maximum number of shifts, illustrating insertion sort's sensitivity to input order.

Test Cases

```
log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
```

```
sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
```

```
assert sorted_log == sorted(log)
```

```
import random
```

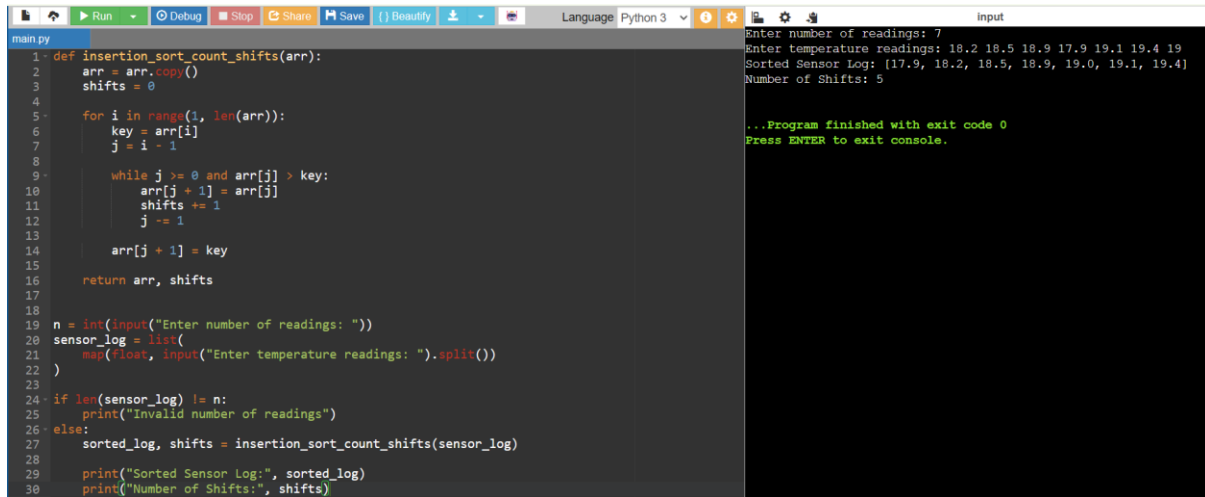
```
shuffled_log = log.copy()
```

```
random.shuffle(shuffled_log)
```

```
_, shifts_random = insertion_sort_count_shifts(shuffled_log)
```

```
print('Nearly-sorted shifts:', shifts_nearly, 'Random shifts:', shifts_random)
```

```
print('All test cases passed!')
```



```
1 def insertion_sort_count_shifts(arr):
2     arr = arr.copy()
3     shifts = 0
4
5     for i in range(1, len(arr)):
6         key = arr[i]
7         j = i - 1
8
9         while j >= 0 and arr[j] > key:
10            arr[j + 1] = arr[j]
11            shifts += 1
12            j -= 1
13
14        arr[j + 1] = key
15
16    return arr, shifts
17
18
19 n = int(input("Enter number of readings: "))
20 sensor_log = list(
21     map(float, input("Enter temperature readings: ").split())
22 )
23
24 if len(sensor_log) != n:
25     print("Invalid number of readings")
26 else:
27     sorted_log, shifts = insertion_sort_count_shifts(sensor_log)
28
29     print("Sorted Sensor Log:", sorted_log)
30     print("Number of Shifts:", shifts)
```

Enter number of readings: 7
Enter temperature readings: 18.2 18.5 18.9 17.9 19.1 19.4 19
Sorted Sensor Log: [17.9, 18.2, 18.5, 18.9, 19.0, 19.1, 19.4]
Number of Shifts: 5

...Program finished with exit code 0
Press ENTER to exit console.